

DATA_MODEL_PLAIN

Helios Data Model — In Plain English

Plain-language companion to `docs/architecture/DATA_MODEL.md` . For the exact spec, read that / its PDF.

In one sentence

Google gives us a firehose of raw "someone clicked something" events, and this document explains how Helios rebuilds that firehose into a few clean, well-organized tables (visits, orders, products, users) that an AI can safely ask questions about.

Why this matters to you

Every number Helios ever reports comes out of these tables. If a table is shaped wrong, the AI gets a wrong answer and can't tell. So this is the load-bearing foundation. The good news: once you understand five or six tables and one or two "gotchas," the whole thing clicks. This guide gets you there fast so you can start building instead of re-reading the spec.

The big ideas, simply

1. GA4 gives us events, not anything useful yet. The public dataset (`bigquery-public-data.ga4_obfuscated_sample_ecommerce` , covering Nov 2020 – Jan 2021) has one row per *event* — one page load, one "add to cart," one purchase. There is no "visit" row, no "order" row, no "customer" row. Helios builds all of those itself.

2. Data flows through five layers, like an assembly line. Each layer does exactly one job:

`raw → staging → intermediate → marts → semantic`

- **raw:** the untouched Google export. We never query it directly.
- **staging:** rename and clean things; unpack the messy nested columns *once*.

- **intermediate**: do the hard business logic — rebuild visits, build the funnel.
- **marts**: the finished, wide tables the product actually uses.
- **semantic**: a thin menu of 47 named metrics the AI picks from (it never writes SQL).

3. Wide tables, no joins at question-time. The finished tables are "wide" — they cram all the descriptive columns (device, country, channel) right onto each row. So when the AI asks "revenue by channel," it just groups one column. No join puzzle for the AI to solve, which is exactly the point.

4. Two things are "entities of record." Everything else hangs off these two:

- **the session** (one visit), identified by `session_key`
- **the transaction** (one completed order), identified by `order_key` (= `transaction_id`)

A third, the **user**, exists but is honestly flagged as approximate (more below).

How it fits together

Think of a **star**: fact tables in the middle, dimension ("dim") tables around the edges describing them.

The session side (a visit):

- `int_ga4__sessionized` — rebuilds the visit Google never shipped, by grouping all events that share (`user_pseudo_id`, `ga_session_id`). The key is `session_key` = `TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))`. This is a **keystone**: get it wrong and everything above is silently wrong.
- `fct_sessions` — the visit's description (device, country, engagement).
- `fct_funnel` — the *same* visit, extended 1:1, carrying the funnel flags and the visit's revenue. **This is the main table the AI queries.**
- `fct_daily_funnel` / `fct_funnel_by_dim` — pre-added-up daily summaries for spotting anomalies and mix shifts.

The order side (a sale):

- `fct_orders` — one clean row per order. Raw GA4 emits duplicate purchase rows, so this de-duplicates by `transaction_id` first.
- `fct_order_items` — one row per product line in an order; where product-level revenue lives.

The dimensions (the describers), shared by everything:

- `dim_date`, `dim_users`, `dim_channels` (exactly 10 channel groups), `dim_items` (the catalog).

The funnel is the heart of it — the path a visitor walks:

```
session_start → view_item → add_to_cart → begin_checkout → add_shipping_info →
add_payment_info → purchase
```

Each step is a `reached_*` flag, built "max-downstream": a visit that reached a later step counts as having reached all earlier ones. That makes the funnel always shrink as you go down, so every step rate stays between 0 and 1 — no impossible "120% conversion."

Easy things to get wrong

- **The `traffic_source` trap.** The event-level `traffic_source` is *first-touch* — the channel that originally got the cookie, stamped on every later visit. Using it as "this visit's channel" mislabels every return visit. Rule: prefer the session-scoped `source / medium`, fall back to `traffic_source` only when those are null.
- **A "user" is a cookie, not a person.** `user_id` is almost always empty, so one human on a phone and a laptop counts as two users. Every user metric (ARPU, new/returning, retention) is a cookie approximation — say so.
- **Don't mix grains.** Count orders at order grain, count sessions at session grain. Summing the wrong measure at the wrong grain double-counts. The semantic layer pins each metric to exactly one table to prevent this.
- **Never store rates, only counts.** Pre-aggregated tables keep counts only; rates are always recomputed as $\text{SUM}(\text{numerator}) / \text{SUM}(\text{denominator})$. This is the Simpson's-paradox defense — averaging pre-divided ratios lies.
- **Money is `_in_usd` only,** and shipping/tax are kept but never folded into revenue or `aov`.
- **Retention is right-censored** at the window's edge — late cohorts haven't had 30 days to return, so `day_30_retention` for them looks low but isn't.

Glossary — the exact words, demystified

- **grain** — what one row means (one event? one visit? one order?). A table's contract.
- **session_key** — the unique fingerprint of one visit; the MD5 hash above.
- **`reached_*` flags** — monotonic funnel markers; "reached this step or any later one."
- **session_conversion_rate** — $\text{purchasing_sessions} / \text{sessions}$; the headline "did visits buy" rate.
- **aov** — average order value = $\text{gross_revenue} / \text{orders}$.

- **revenue_per_session** — revenue / sessions; equals $\text{session_conversion_rate} \times \text{aov}$.
- **conformed dimension** — one shared describer table (date, user, channel, item) used identically everywhere, so slices line up.
- **mart** — a finished, query-ready table.
- **keystone** — a model that fails silently if wrong (`int_ga4__sessionized` , `int_ga4__funnel_steps` , revenue dedup); test these hardest.
- **dedup by transaction_id** — collapse Google's duplicate purchase rows to one before summing money.

When to open the real doc

Open `pdf/docs/architecture/DATA_MODEL.pdf` when you need the exact column lists, the real SQL for each model, the full ER diagram, or the ownership/steward rules. This companion is the map; the spec is the territory.